

**SE1012 Programming Methodology Assignment Implementing a
Two-Player Yahtzee Game in C**

Report

IT24103479
M.P.A Withanage

Introduction

Yahtzee is a dice game and the objective of the game is to score points by rolling dice to make certain combinations. The game consists of thirteen rounds and each player has up to three rolls per turn, with the option to keep or re-roll certain dice to improve their chances of achieving a high-scoring combination.

In this assignment, a simplified version of the classic game is implemented using C language. The game is a two-player version where humans compete against the computer. The computer opponent is designed to challenge the player through strategic decision-making. The computer's strategy emulates a realistic opponent by prioritizing high-scoring categories, choosing optimal dice to keep based on the roll outcomes.

Computer Player Strategy

1. Dice-keeping

The computer player prioritizes consistent, high-scoring categories like Small Straight, Large Straight, and Full House due to their fixed scores and relatively high odds compared to categories like Yahtzee. This approach helps secure stable points early in the game.

When dice of a particular value appear in counts of 3 or more, that value is prioritized. Repeated attempts for straights can sometimes be unsuccessful, leading to missed scoring opportunities in other categories. Once the Small and Large Straights categories are achieved, priority is given to the Full House category. For categories like Ones, Twos, and Three of a Kind, the computer keeps the most frequent dice value, which improves the chances of efficiently meeting these category requirements.

Using flags

Flags like “mostcountedkeeping”, “straightskeeping”, and “full-house” were implemented to prevent the computer player from attempting to keep dice combinations for multiple categories simultaneously.

mostcountedkeeping:

This flag is set when the computer decides to keep the most frequently rolled dice to aim for categories such as Three of a Kind, Four of a Kind, or Yahtzee. Once this flag is active, it prevents the computer from focusing on other categories like straights in the same turn. This helps the computer avoid a conflicting strategy where it might attempt both a high-frequency match and a sequence simultaneously.

straightskeeping:

This flag is activated when the computer begins keeping dice for sequential categories, like Small Straight or Large Straight. Activating this flag stops the computer from switching its focus mid-turn to other categories that require matching values, ensuring that the computer maintains consistency in its attempts to complete a straight.

If small straight, large straight , full house and Yahtzee category met

the computer prioritizes high-scoring categories like Small Straight, Large Straight, and Full House. If any of these combinations are met, the computer immediately selects the category and ends its turn, avoiding further dice-keeping. This strategy ensures the computer maximizes its score by focusing on reliable, high-value combinations early in the game.

2. Category choosing

The computer prioritizes categories with fixed scores, such as Small Straight, Large Straight, Full House, and Yahtzee. If any of these combinations appear, the highest-scoring category is selected. If none of these categories appear, the highest-scoring available category is chosen. If categories are used or scored zero by the third turn, the computer will select any available category, even if its score is zero.

3.Fallback Strategy

The computer prioritizes high-scoring categories like Small Straight and Large Straight. Although Yahtzee offers the highest score, it is deprioritized because the odds of achieving this combination are smaller. If the dice don't match these combinations, the computer falls back to categories such as Four of a Kind, Three of a Kind, N's (ones, twos, threes, etc.), and Yahtzee.

2. Challenges and Solutions

Implementing Dice-Keeping Strategy

Challenge: Designing a dice-keeping logic that mimics a skilled player's choice of combinations, particularly for categories with fixed scores.

Solution: Flags were used to help the computer prioritize Small Straight, Large Straight, and Full House over other combinations. Once a category was selected, the flag system prevented the computer from keeping additional dice for conflicting categories in the same turn.

Balancing Category Selection

Challenge: Prioritizing categories with fixed scores while still maximizing potential points in other categories.

Solution: The computer first attempted to achieve Small Straight, Large Straight, or Full House, switching to fallback options like Four of a Kind or Three of a Kind if these were not feasible. A scoring hierarchy helped the computer make a choice when multiple combinations appeared.

Managing Low-Probability Combinations

Challenge: Balancing the difficulty of achieving high-value categories like Yahtzee with the frequency of lower-scoring categories.

Solution: The computer deprioritized Yahtzee due to its lower probability and, when necessary, selected any available category, even if the score was zero, to ensure continuous gameplay progression.

User-Friendly Interaction

Challenge: Ensuring the game remains user-friendly and convenient while keeping the code streamlined.

Solution: A scoreboard is displayed at each turn, allowing the user to track scores and make informed decisions. The interface also highlights chosen categories and displays round-by-round scores, providing clarity on current standings.

3.Reflection on Implementation

Implemented Features and Their Effectiveness

Modular Game Design

Modularizing the game proved beneficial for both functionality and simplicity, enabling the use of shared functions across human and computer players. Functions such as scoreboard, category, and rollingdice streamlined gameplay by reducing redundancy and simplifying the code structure.

Computer Decision-Making

The computer player's decision-making process effectively prioritized high-scoring categories like Small Straight, Large Straight, and Full House. This scoring prioritization helped the computer achieve consistent scores without over-relying on difficult-to-achieve categories, such as Yahtzee. This design allowed the computer to dynamically adapt its decisions based on available dice combinations, keeping gameplay competitive.

Features Not Implemented and Reasons Why

Inability to Replace Kept Dice:

Once a die is kept, it cannot be replaced later in the turn. This feature was not implemented because it would require complex logic, with the computer needing to reassess the game state and adjust its choices based on new dice rolls.

Limited Navigation between Dice Keeping and Category Selection:

Once the game moves to the category selection phase, neither the computer nor the user can return to the dice-keeping phase. This choice was made to simplify gameplay. Allowing phase-switching would require the user to repeatedly enter "0" to move forward, adding unnecessary steps that could disrupt the game flow. Using "0" as the transition input keeps the design straightforward, avoiding the need for complex input handling or type casting.

Enter the number you want to keep (1-6) or type 0 to move on:

Select category (1-13) or 0 to move on:

Bonus

The bonus feature, such as the Upper Section or Yahtzee Bonus, was not implemented due to its complexity, requiring extra logic to track and calculate eligibility dynamically.

Future implements

- **Enhancing Computer Decision-Making Skills:** Improving the computer's logic to track unscored categories each turn and adjust its dice-keeping to target them. For example, if Large Straight hasn't been scored yet, it would focus on keeping sequential dice; for Four of a Kind, it would keep the highest-count dice. This adjustment would increase the computer's scoring opportunities and make it a more adaptable, strategic opponent.
- **User-Friendly Dice Interaction:** Future improvements could allow players to click on dice to keep them, simplifying gameplay and making it more interactive.
- **Flexible Dice Replacement:** Adding a feature to replace kept dice between rolls would offer strategic flexibility, though this would increase the complexity of decision-making logic.
- **Adding bonus feature:** The addition of a bonus feature, such as the Upper Section or Yahtzee Bonus, will involve tracking the player's scores and determining when they qualify for additional points. This will add complexity to the game and enhance the overall gameplay experience by providing more opportunities for strategy and higher scores.

